



TEK YÖN

DATAFLOWX · STAJ ARAŞTIRMA & UYGULAMA PROJESİ

DiodeLab

Veri diyotu kavramını anlamak, ağ katmanları üzerinden mantığını çözmek ve **fiziksel diyot olmadan** tek-yönlü veri akışını **Docker üzerinde** kurup, kendi yazdığın yazılımla uçtan uca test etmek.

Data Diode

Docker & Compose

UDP + FEC

Go / Python

Ağ Katmanları

Uygulamalı Yazılım

Tür: Araştırma + yazılım
geliştirme laboratuvarı

Seviye: Üniversite /
temel kodlama

Çıktı: Çalışan sanal diyot + Tx/Rx
yazılımı + test raporu

1 · Bu Doküman Ne Sunuyor?

Bu proje iki ayaklı: (1) **anlamak** — veri diyotu neden var, hangi ağ kuralına dayanıyor, gerçekte nasıl çalışıyor; (2) **yapmak** — fiziksel donanım olmadan tek-yönlü bir hattı **Docker** ortamında kurmak ve üzerinde **kendi yazacağın** gönderici/alıcı yazılımıyla dosya taşımak. Ağırılık, yazılımı yazıp çalıştırmakta.

Sonunda üç somut çıktın olacak: veri diyotunun mantığını ağ katmanı seviyesinde anlatabilmek; Docker’da tek-yönlülüğü kurup *kanıtlamak*; ve geri kanal (ACK) olmayan bir hatta veriyi güvenilir taşıyan çalışan bir yazılım.

Net sınır

Gerçek donanım veri diyotu **kullanmayacaksın** ve gerek de yok. Fiziksel diyotun tek-yönlülük garantisini yazılım + ağ kurallarıyla *modelleyeceğiz*. Amaç üretim güvenliği değil, kavramı ve mimariyi **uygulayarak** öğrenmek.

2 · Veri Diyotu Nedir? (Kısa)

Elektronikteki diyot akımı tek yöne geçirir. **Veri diyotu** da veriyi *yalnızca bir yöne* akıtan, geri yönde tek bir bit’in bile geçmesini engelleyen bir cihazdır (tipik olarak tek yönü bağlı fiber: bir uçta yalnız verici, diğerinde yalnız alıcı). Amaç: yüksek güvenli bir ağdan (kritik altyapı, SCADA/OT, sınıflandırılmış ağ) veriyi *dışarı çıkarmak* ama **içeri hiçbir şey sızdırmamak**. Geri kanal mimari olarak yok olduğundan uzaktan komut/sızma geri yolu da yoktur.

ARAŞTIRILACAK SORULAR

- “Air gap” nedir; veri diyotu ondan nasıl ayrışır (“air-gap’i bozmadan tek yönlü veri çıkarmak”)?
- OT/IT ayrımı ve SCADA’da tek-yönlülük neden kritik? Nereelerde kullanılır (log, telemetri, yedek)?

3 · Ağ Katmanları Merceğinden Diyet

İşin kalbi taşıma katmanında: **TCP** çift yönlü el sıkışma (SYN/ACK) ve onay (ACK) gerektirir; geri yol yoksa *çalışamaz*. Bu yüzden diyotlar onay beklemeyen **UDP** (ya da özel tek-yönlü protokol) kullanır. Uygulama katmanında ise her iki uçta birer **proxy** vardır: gönderen taraf normal veriyi (dosya) alıp tek-yönlü akışa çevirir; alan taraf toplayıp tekrar dosyaya döndürür. Senin yazacağın yazılım tam olarak bu iki proxy'dir.

Katman	Diyot açısından
L1 Fiziksel	Gerçekte tek-yönlülük burada zorlanır (tek yönlü fiber). Biz bunu ağ kuralıyla modelleyeceğiz.
L4 Taşıma	TCP kırılır (ACK dönemez) → UDP kullanılır. “Gönder ve unut”; bedeli: kayıpları fark edemezsin.
L7 Uygulama	Tx proxy (parçalar & gönderir) + Rx proxy (toplar & birleştirir). Kodlayacağın kısım.

ARAŞTIRILACAK SORULAR

- TCP üçlü el sıkışması adım adım nasıl işler; hangi adım tek-yönlü hatta imkânsız olur?
- UDP neyi kazandırır/kaybettirir? IP için gereken ARP çift yönlüdür — tek-yönlü hatta bu nasıl sorun çıkarır, nasıl aşılar (statik ARP)?

4 · ACK Olmadan Güvenilirlik (Yazılımın Çözeceği Problem)

Geri kanal olmadığı için “alamadım, tekrar gönder” diyemezsin. Veriyi baştan **kayba dayanıklı** göndermelisin — bu, yazacağın yazılımın çekirdek mühendislik problemi:

- **Parçalama + sıra no + hash:** dosya sabit boyutlu parçalara bölünür; her parçaya indeks ve `sha256` eklenir; alıcı eksiği/bozuğu tespit eder.
- **Fazlalık (redundancy):** aynı parçayı birden çok kez göndermek — basit ama işe yarar ilk adım.
- **FEC (Forward Error Correction):** fazladan “onarım/parity” parçaları; bazıları kaybolursa bile kalanlardan orijinal yeniden inşa edilir (erasure coding fikri).

ARAŞTIRILACAK SORULAR

- Erasure coding’de “N parçadan herhangi K tanesi yeter” fikri ne demek? En basit FEC’i nasıl kurarsın?
- Hangi kayıp oranına kadar kurtarabilmelisin; bunu nasıl ölçüp ayarlarsın?
- Bütünlük (hash) ile güvenilirlik (kayıbı telafi) aynı şey mi? Hangisi neyi garanti eder?

5 · Çalışma Ortamı — Docker (Önerilen) & netns (Alternatif)

Fikir sabit: iki yalıtılmış “host” yarat, aralarına bir bağlantı koy, **geri yöndeki tüm trafiği düşür (DROP)**. Böylece fiziksel tek-yönlülüğü ağ kuralıyla modellersin. Ortam için iki yolun var; **Docker’ı öneriyoruz** — taşınabilir, tek komutla kur/yık, ekosistemle uyumlu.

5.1 · Docker ile kurulum (önerilen, detaylı)

İki container: **tx** (kaynak) ve **rx** (hedef), ortak özel bir bridge ağında. Proxy yazılımların bu container’ların içinde çalışır. Dosyaları host’tan izlemek için birer klasör bağlarsın (**./data/in** → tx, **./data/out** → rx).

```
# docker-compose.yml — kavramsal iskelet
services:
  tx:
    build: .
    command: ["/app/tx", "--in=/data/in", "--target=rx:9000"]
    volumes: [".:/data/in:/data/in"]
    networks: [diode-net]
    cap_add: [NET_ADMIN] # iptables/tc için gerekli
  rx:
    build: .
    command: ["/app/rx", "--listen=:9000", "--out=/data/out"]
    volumes: [".:/data/out:/data/out"]
    networks: [diode-net]
    cap_add: [NET_ADMIN]
networks:
  diode-net: { driver: bridge }
```

Tek-yönlülüğü zorlamak

Docker bridge varsayılan olarak *çift yönlüdür*. Tek-yönlülüğü sen dayatacaksın. İki katmanlı düşün:

- **Yazılım seviyesi:** Tx yalnızca *gönderir* (dinleyen port açmaz); Rx yalnızca *dinler*. Yani uygulama zaten tek yönlü.
- **Ağ seviyesi (kanıt):** rx içinde bir **iptables** kuralıyla **rx → tx** yönündeki tüm çıkışı **DROP** et. Böylece “niyet” değil, *kural* tek-yönlülüğü garanti eder ve bunu test edebilirsiniz.

```
# rx container'ında giden (tx'e dönen) trafiği düşür — tek yönü kanıtlarla
iptables -A OUTPUT -d <tx_ip> -j DROP

# tx tarafında kayıplı hattı simüle et (FEC'i test etmek için)
tc qdisc add dev eth0 root netem loss 5%
```

Neden Docker?

Tüm ortam **docker compose up** ile ayağa kalkar, **down** ile temizlenir. Kurulum kodun içinde (Dockerfile + compose) yaşar, “bende çalışıyordu” sorununu ortadan kaldırır. Şirket ürünleri de Docker ile paketlenir — gerçek üretim refleksi.

5.2 • Alternatif: Linux network namespaces (hafif)

Docker istemezsen tek makinede `ip netns` + `veth` ile de kurabilirsin: iki namespace (`source` / `sink`), veth çifti, ve `nftables` ile geri yön DROP. Daha az bağımlılık ama ARP/IP detaylarıyla biraz daha çok uğraştırır. Docker'ı seçersen bunu opsiyonel bir ek olarak deneyebilirsin.

ARAŞTIRILACAK SORULAR

- Container'a neden `NET_ADMIN` yetkisi gerekiyor? `tc` / `iptables` bu yetki olmadan neden çalışmaz?
- Tek-yönlülüğü nasıl *kanıtlarsın*? (İpucu: `rx` 'ten `tx` 'e `ping` /UDP dene → geçmemeli; ileri yön → geçmeli. `tcpdump` ile göster.)
- Docker bridge ağında container'lar birbirini isimle (`rx` , `tx`) nasıl bulur (DNS)? IP'ler nasıl atanır?
- `tc netem` ile kayıp/gecikme/karışık-sıra nasıl enjekte edilir? FEC'ini sınamak için hangi senaryoları kurarsın?

6 • Mimari Genel Bakış



İki program yazacaksın: `tx` ve `rx`. Aralarında **hiçbir geri mesaj** yoktur — Rx, “aldım/ almadım” diyemez. Tüm zeka, tek yönde gönderilen paketlerin kendi başlarına yeterince bilgi taşımasında.

7 · Yazılım: Ne Yazacaksın, Nasıl?

Bu bölüm projenin ağırlık merkezi. İki komut satırı programı (`tx` ve `rx`) ve bunların paylaştığı birkaç küçük modül geliştireceksin. Aşağıda **ne** yazacağın, hangi parçalara böleceğin ve **hangi sırayla** ilerleyeceğin var. Algoritmalar sözde-kod (pseudocode) — birebir kopyalanacak kod değil, *senin* dolduracağın iskelet.

7.1 · Önerilen proje yapısı

Her parçanın tek bir işi olsun; böylece ayrı ayrı test edebilirsin. (Go örneği; Python’da paket yerine modül düşün.)

```
diodelab/
cmd/tx/main.go      # kaynak proxy: CLI, config, akışı yönetir
cmd/rx/main.go      # hedef proxy: CLI, dinler, dosyayı yazar
internal/frame/     # paket (chunk) formatı: serialize / deserialize + hash
internal/chunker/    # dosya <-> parçalar
internal/transport/  # UDP gönder / al (ince sarmalayıcı)
internal/fec/        # (opsiyonel) parity üret / kurtar
internal/reassembly/ # gelen parçaları biriktir, tamlığı takip et
scripts/            # docker-compose.yml, Dockerfile, netem/iptables
data/in data/out     # test dosyaları
README.md
```

7.2 · Protokol: bir paket (frame) neye benzer?

Sıra bozulabilir ve paketler kaybolabilir; bu yüzden **her UDP paketi kendi başına anlamlı** olmalı. Bir “manifest” paketi transferi tanıtır, “chunk” paketleri veriyi taşır, “end” paketi bitiş işaretler. Geri kanal olmadığından manifest ve end paketlerini *birkaç kez tekrarlar*.

```
ORTAK BAŞLIK: { transferId, type: MANIFEST|CHUNK|END }
MANIFEST : { fileName, fileSize, totalChunks, chunkSize, sha256(dosya) }
CHUNK    : { index, isParity, payloadLen, payload, sha256(payload) }
END      : { totalChunks }
```

▣ ARAŞTIRILACAK SORULAR

- Serileştirme için JSON (kolay, okunur ama şişkin; payload’ı base64 gerekir) mi, binary (kompakt, biraz zor) mı? Ne zaman hangisi?
- UDP paket boyutu ne olmalı? MTU’yu aşarsan ne olur (IP fragmentation) ve bu neden kayıp riskini artırır?
- Rx bir paketin hangi transfere ait olduğunu nasıl anlar? `transferId` ’yi nasıl üretirsin (çakışmasın)?

7.3 · Kaynak proxy (tx) — sorumluluk & algoritma

Görevi: bir dosyayı, geri kanal olmadan karşıya “yeterince” ulaşacak biçimde tek yönde yaymak.

```

tx(inputFile, targetAddr, opts):
    data      = read(inputFile)
    fileHash  = sha256(data)
    chunks    = chunker.split(data, opts.chunkSize)    # [payload0, payload1, ...]
    transferId= newId()

    manifest = Manifest{transferId, name, size, len(chunks), chunkSize, fileHash}
    repeat opts.manifestRepeat times: udp.send(serialize(manifest))    # kaybolmasın

    frames = []
    for i, p in chunks: frames.add(Chunk{transferId, i, false, p, sha256(p)})
    if opts.fec: frames += fec.makeParity(chunks)    # opsiyonel onarım paketleri

    for f in frames:
        repeat opts.redundancy times: udp.send(serialize(f))
        sleep(opts.pacing)    # tampon taşmasın (backpressure yok!)

    repeat opts.manifestRepeat times: udp.send(serialize(End{transferId, len(chunks)}))

```

▣ ARAŞTIRILACAK SORULAR

- Neden paketler arasına küçük bir bekleme (`pacing`) koymak gerekebilir? Çok hızlı gönderirsen ne kaybolur?
- Manifest'i kaç kez tekrarlamalı? Tekrar hiç gelmezse Rx transferi nasıl yine de kurtarabilir?
- Go'da UDP soketi (`net.DialUDP` / `WriteToUDP`) veya Python `socket(AF_INET, SOCK_DGRAM)` nasıl kullanılır?

7.4 · Hedef proxy (rx) — sorumluluk & algoritma

Görevi: gelen paketlerden dosyayı yeniden kurmak; eksik/bozuğu tespit etmek; “bitti mi?” kararını geri kanal olmadan vermek.

```

rx(listenAddr, outDir, opts):
    sessions = {}                                # transferId -> durum
    loop:
        pkt = udp.recv(timeout=opts.idle)
        if timeout: sweepStaleSessions(); continue
        f = deserialize(pkt)
        s = sessions.getOrCreate(f.transferId)

        switch f.type:
            MANIFEST: s.setMeta(f); s.alloc(f.totalChunks)
            CHUNK:
                if sha256(f.payload) != f.sha256: drop(f); continue # bozuk paket
                if f.isParity: s.parity.add(f) else s.chunks[f.index]=f.payload
            END: s.sawEnd = true

        if s.hasMeta and s.isComplete():          # tüm veri parçaları geldi
            finish(s)
        elif s.hasMeta and s.sawEnd and s.canRecoverWithFEC():
            s.recoverMissing(); finish(s)

    finish(s):
        data = join(s.chunks)
        if sha256(data) == s.fileHash: write(outDir/s.name); log("OK")
        else: quarantine(s); log("BOZUK/EKSIK")

```

ARAŞTIRILACAK SORULAR

- Rx bir transferin “bittiğini” geri kanal olmadan nasıl bilir? (tüm parça sayısı + END + *idle timeout* kombinasyonu.)
- Manifest hiç gelmediyse ne yaparsın? Parçalar var ama toplam sayı bilinmiyorsa?
- Eksik/onarılamaz dosyayı ne yapmalı — sil, karantinaya al, logla? Kullanıcıya nasıl bildirir?
- Aynı transfer için birden çok Rx örneği (idempotency) veya yeniden başlatma durumunda ne olur?

7.5 • FEC / kurtarma katmanı (opsiyonel ama öğretici)

Önce **redundancy** (her parçayı N kez gönder) ile başla — basit ve hemen çalışır. Sonra gerçek FEC’e geç: en sade biçimi **XOR-parity**’dir — K parçanın XOR’u bir parity paketi verir; K parçadan *herhangi biri* kaybolursa parity + kalanlarla geri hesaplanır. Daha ileri: Reed-Solomon (hazır kütüphane).

ARAŞTIRILACAK SORULAR

- XOR-parity ile tek bir kayıp parça nasıl geri hesaplanır? Neden aynı grupta *iki* kayıpta çalışmaz?
- Ne kadar parity = ne kadar dayanıklılık, karşılığında ne kadar fazladan bant genişliği? Bu dengeyi nasıl kurarsın?

7.6 · Yazılımı nasıl test edersin?

- **Birim test:** `chunker` (böl→birleştir aynı veriyi vermeli), `frame` (serialize→deserialize kayıpsız), `fec` (bir parça sil→kurtar).
- **Entegrasyon:** kayıpsız hatta dosya %100 birebir gelmeli (`sha256` eşit).
- **Kaos/dayanıklılık:** `tc netem loss` ile %1/%5/%10 kayıpta başarı oranını ölç; FEC açık/kapalı karşılaştır; sonuçları tabloya/grafiğe dök.
- **Tek-yönlülük testi:** `rx→tx` bağlantısının kurulamadığını göster (kanıt).

Geliştirme sırası (yatay dilimler)

1) Önce en küçük çalışan şey: tek parçalık dosyayı UDP ile gönder-yaz. 2) Sonra çok parçalı + hash. 3) Sonra Docker'a taşı + tek-yönlülüğü kanıtla. 4) Sonra netem ile kayıp + redundancy. 5) En son FEC. Her adımda çalışan bir şey olsun.

8 · Gereksinimler

Zorunlu (çekirdek) önce; **Opsiyonel** vakit kalırsa.

No	Gereksinim	Öncelik
D1	Docker Compose ile <code>tx</code> ve <code>rx</code> container'ları tek komutla ayağa kalkar	Zorunlu
D2	Tek-yönlülük iptables ile zorlanır ve testle kanıtlanır (ileri UDP geçer; <code>rx→tx</code> kapalı)	Zorunlu
D3	<code>tx</code> : dosyayı parçalar, manifest+chunk+end paketlerini UDP ile gönderir	Zorunlu
D4	<code>rx</code> : parçaları toplar, per-chunk ve dosya hash'ini doğrular, dosyayı eksiksiz yazar	Zorunlu
D5	Kayıpsız hatta transfer %100 birebir (kaynak ve hedef <code>sha256</code> eşit)	Zorunlu
D6	Kritik modüller için birim testler (chunker, frame)	Zorunlu
D7	Redundancy: parça tekrarıyla düşük kayıpta başarıyı artırma + netem ile ölçüm	Opsiyonel
D8	FEC (XOR-parity veya Reed-Solomon) ile daha yüksek kayıpta kurtarma	Opsiyonel
D9	Dizin/çoklu dosya transferi + ilerleme & sonuç özeti	Opsiyonel
D10	netns (Docker'sız) kurulumunun da script'lenmesi	Opsiyonel

Teknik tercihler

Konu	Beklenti
Dil	Go önerilir (tek binary, ekosistemle uyumlu); Python da kabul. Seçimini gerekçelendir.
Ortam	Docker + Compose (birincil). İçeride <code>iproute2</code> / <code>iptables</code> kurulu olsun; <code>NET_ADMIN</code> .
Tekrarlanabilirlik	Her şey tek komutla kur/yık; kurulum kodda (Dockerfile+compose+script) yaşar.
Kanıt kültürü	Her iddia ("tek yönlü", "kayba dayanıklı") bir testle gösterilir; sonuçlar kaydedilir.
Sürüm kontrol	Git + anlamlı commit'ler + kur/çalıştır anlatan <code>README</code> .

9 · Yol Haritası (Aşamalar)

Gün gün değil, **aşama aşama**. Her aşama sonunda gösterilebilir somut bir sonuç olmalı.

Aşama 0 — Kavramı sök (başlangıç)

Bölüm 2-4 araştırma sorularını yanıtla; “neden TCP çalışmaz, UDP+FEC nasıl çözer” notunu yaz. Dil/serileştirme kararını gerekçelendir.

Aşama 1 — En küçük çalışan transfer (Docker’sız, localhost) (erken)

Tek makinede: `tx` bir dosyayı UDP ile `rx`’e gönderir, `rx` yazar. Önce tek parça, sonra çok parça + hash. İlk somut çıktı.

Aşama 2 — Docker ortamı + tek-yönlülük kanıtı (orta)

Dockerfile + compose ile `tx / rx` container’ları. iptables ile geri yönü kapat; `tcpdump / ping/UDP` testiyle tek-yönlülüğü **kanıtla**.

Aşama 3 — Kaybı devreye al, dayanıklılığı ölç (orta-geç)

`tc netem` ile kayıp enjekte et; ne bozuluyor gözlemler. Önce redundancy, sonra FEC ekleyip belirli orana kadar kurtarmayı sağla. Kayıp-oranı vs. başarı grafiği çıkar.

Aşama 4 — Cila, test & doküman (geç)

Birim testler, hata durumları (eksik/bozuk dosya), loglama, README, kur/yık script’leri. Test sonuçlarını derle, teknik dokümanı tamamla.

Aşama 5 — İleri hedefler & sunum (kalan süre)

Opsiyoneller (dizin transferi, izleme özeti, netns kurulumu). Demo senaryosu + sunum.

Altın kural

Her aşamada kod çalışır durumda olsun; her iddiayı testle kanıtla; her kararın *neden*’ini yaz. Takıldığında en geç yarım günde mentordan yardım iste.

10 · Teslimatlar & “Bitti” Tanımı

1. **Git repo:** `tx / rx` kaynak kodu + modüller + Dockerfile/compose + script’ler + `README`.
2. **Çalışan demo:** `docker compose up` → bir dosya tek-yönlü hattan eksiksiz karşıya geçer; geri yönün kapalı olduğu kanıtlanır.
3. **Test & ölçüm raporu:** tek-yönlülük kanıtı; kayıp oranına karşı başarı (redundancy/FEC açık-kapalı) sonuçları.
4. **Teknik doküman (3-5 sayfa):** diyot mantığı, ağ katmanını analizi, yazılım mimarisi & protokol kararları ve *neden*’leri, araştırma sorularının cevap özetleri.
5. **Kısa sunum:** ne yaptın, neyi neden seçtin, nerede zorlandın, daha çok vaktin olsa ne eklerdin.

Bir özellik ne zaman “bitmiş” sayılır?

- İddia edilen davranış bir **testle gösterilebiliyor** (özellikle tek-yönlülük ve kayıp dayanıklılığı).
- `docker compose up` ile başkası da aynı sonucu üretebiliyor; README yeterli.
- Hata durumları makul: eksik/bozuk dosya sessizce kaybolmuyor, loglanıyor.

11 · Değerlendirme Kriterleri (mentor için)

Not “kaç özellik” değil, **kavramsal anlayış + yazılım/mühendislik olgunluğu** üzerinedir.

Kriter	Neye bakılır	Ağırlık
Çalışan yazılım (çekirdek)	D1-D6 uçtan uca çalışıyor; Docker’da transfer + tek-yönlülük kanıtlanmış mı?	30%
Yazılım mimarisi & kod kalitesi	Modül ayrımı, protokol tasarımı, hata yönetimi, birim testler, git hijyeni.	25%
Kavramsal anlayış	Diyot mantığı, TCP/UDP ikilemi, ACK’sız güvenilirliği doğru anlatabiliyor mu?	20%
Kanıt & test kültürü	İddialarını ölçüm/testle destekliyor; kayıp senaryolarını kurgulamış mı?	15%
Doküman & iletişim	README, teknik doküman, sunum netliği; ilerleme paylaşımı.	10%

12 · Başlangıç Kaynakları

- **Kavram:** data diode / unidirectional gateway; air-gap; OT/IT ayrımı; SCADA güvenliği temelleri.
- **Ağ:** OSI katmanları; TCP el sıkışma & ACK; UDP; ARP; MTU & IP fragmentation.
- **Docker:** Dockerfile, `docker compose`, bridge ağ & container DNS, `cap_add: NET_ADMIN`, volume bağlama.
- **Ağ araçları (container içinde):** `iptables`, `tc / netem`, `tcpdump` (paketleri görmek).
- **Güvenilirlik:** Forward Error Correction; XOR-parity; erasure coding / Reed-Solomon; checksum/hash (SHA-256).
- **Programlama:** Go `net` (UDP socket) + dosya G/Ç + `testing`; ya da Python `socket` / `hashlib` / `unittest`.

Son söz

Bu proje bilerek boşluklar bırakıyor: serileştirmeyi sen seçeceksin, FEC oranını sen ayarlayacaksın, “bitti mi?” kararını sen vereceksin. Asıl öğrenme bu boşluklarda. Bir yol seç, **neden seçtiğini yaz**, testle kanıtla, ilerle. Başarılar! ►►►